

Proyecto Biometría y Medio Ambiente

- Sprint 0

Víctor Morant Faus

1. Lógica de negocio.....	2
Esquema y resumen.....	2
Propósito.....	2
Funciones principales.....	2
1. getUltimaMedicion(db, callback).....	2
2. getHistorialMediciones(db, cantidad, callback).....	2
Uso.....	2
Ventajas.....	3
2. Ingeniería inversa.....	3
2.1 Sparkfun.....	3
2.1.1. HolaMundoIBeacon.ino.....	3
2.1.2. LED.h.....	4
2.1.3. PuertoSerie.h.....	5
2.1.4. Medidor.h.....	6
2.1.5. Publicador.h.....	7
2.1.6. EmisoraBLE.h.....	8
2.1.7. ServicioEnEmisora.h.....	9
2.1.8 Resumen de relaciones.....	9
2.2 Android.....	10
2.2.1 MainActivity.java.....	10
2.2.2 TramalBeacon.java.....	11
2.2.3 Utilidades.java.....	13
2.2.4 Lógica Fake (FirebaseHelper.java).....	14
Resumen de relaciones y flujo.....	15
3. Diseño BBDD.....	17
4. Web.....	18
4.1 Diseño de la App Web.....	18
4.2 Visual de la App Web.....	19
5. GitHub.....	19
6. Trello.....	19
7. Pruebas del funcionamiento.....	20
7.1 Tests automáticos.....	20
Objetivo de los tests.....	20
Estructura de las pruebas.....	20
7.2 Flujo de funcionamiento.....	21

1. Lógica de negocio

Al hacer uso de una Base de Datos noSQL, la lógica de negocio de mi aplicación corresponde al archivo en la parte del Cliente Web (firebaseService.js) que se encarga de hacer la petición de los datos a Firebase.

Esquema y resumen

Propósito

Centraliza la lógica de consulta de datos a Firebase Realtime Database para la aplicación web, facilitando la reutilización y el mantenimiento del código.

Funciones principales

1. **getUltimaMedicion**(db, callback)

- Consulta: Obtiene la última medición registrada en la base de datos.
- Lógica:
 - Realiza una consulta a la rama "mediciones", ordenando por el campo timestamp y limitando el resultado a 1 elemento.
 - Cuando recibe los datos, llama al callback con el último objeto de medición (o null si no hay datos).

2. **getHistorialMediciones**(db, cantidad, callback)

- Consulta: Obtiene las últimas cantidad de mediciones registradas.
- Lógica:
 - Realiza una consulta a la rama "mediciones", ordenando por timestamp y limitando el resultado a cantidad de elementos.
 - Convierte el objeto recibido en un array y lo ordena de más reciente a más antiguo.
 - Llama al callback con el array de mediciones (o array vacío si no hay datos).

Uso

- Ambas funciones reciben el objeto db (instancia de la base de datos) y un callback que procesa los datos recibidos.
- Permiten separar la lógica de negocio (consultas a Firebase) del resto de la aplicación (presentación y manipulación de datos).

Ventajas

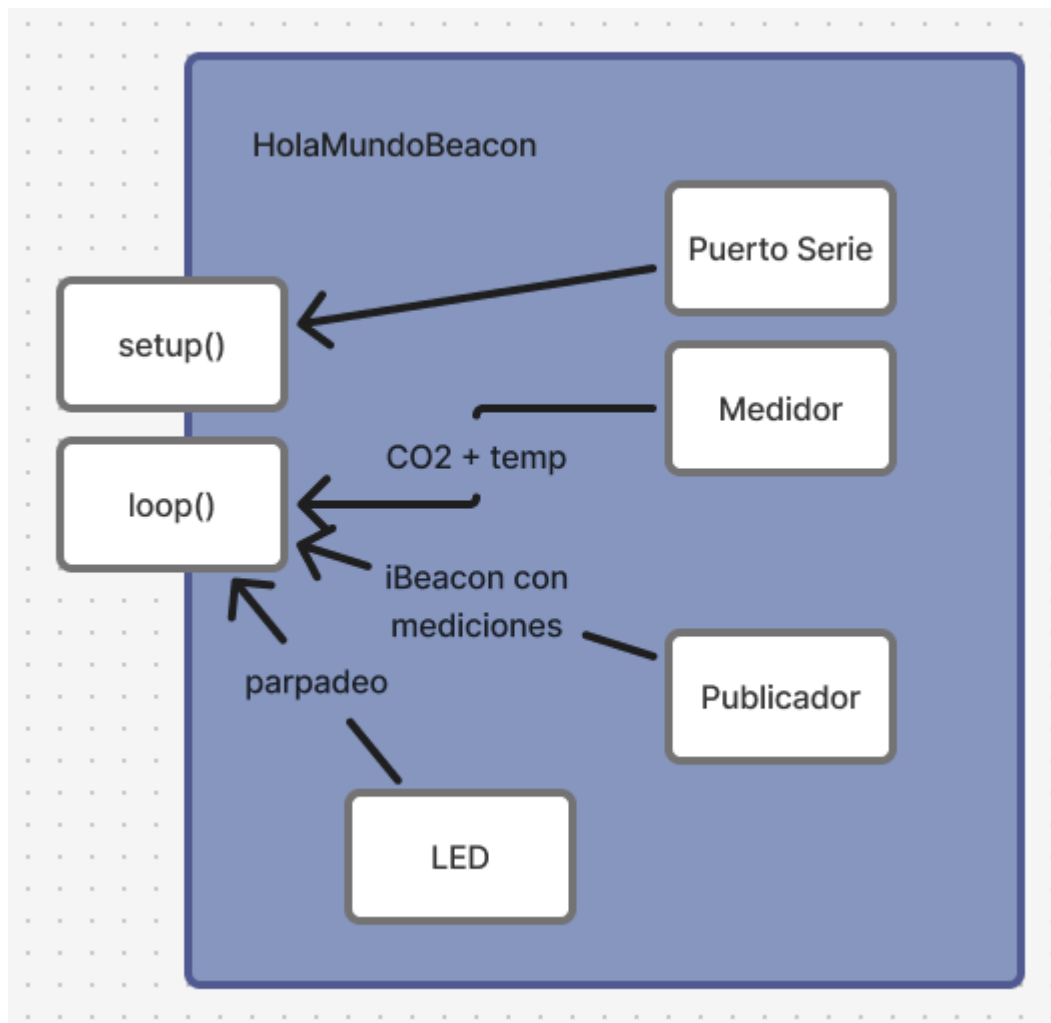
- Modularidad: Facilita el mantenimiento y la reutilización.
- Claridad: El código de la interfaz no se mezcla con la lógica de acceso a datos.
- Escalabilidad: Puedes añadir más funciones de consulta en este archivo si lo necesitas.

2. Ingeniería inversa

A continuación se detalla el funcionamiento de los programas proporcionados para este sprint. Se describe el propósito, relaciones y funciones principales de cada uno de los archivos, además de un resumen general relacionando todos estos con el funcionamiento del programa.

2.1 Sparkfun

2.1.1. HolaMundoIBeacon.ino



Propósito:

Archivo principal del firmware Arduino. Orquesta la inicialización de hardware, la lógica de medición y la publicación de datos vía Bluetooth LE (iBeacon).

Funciones principales:

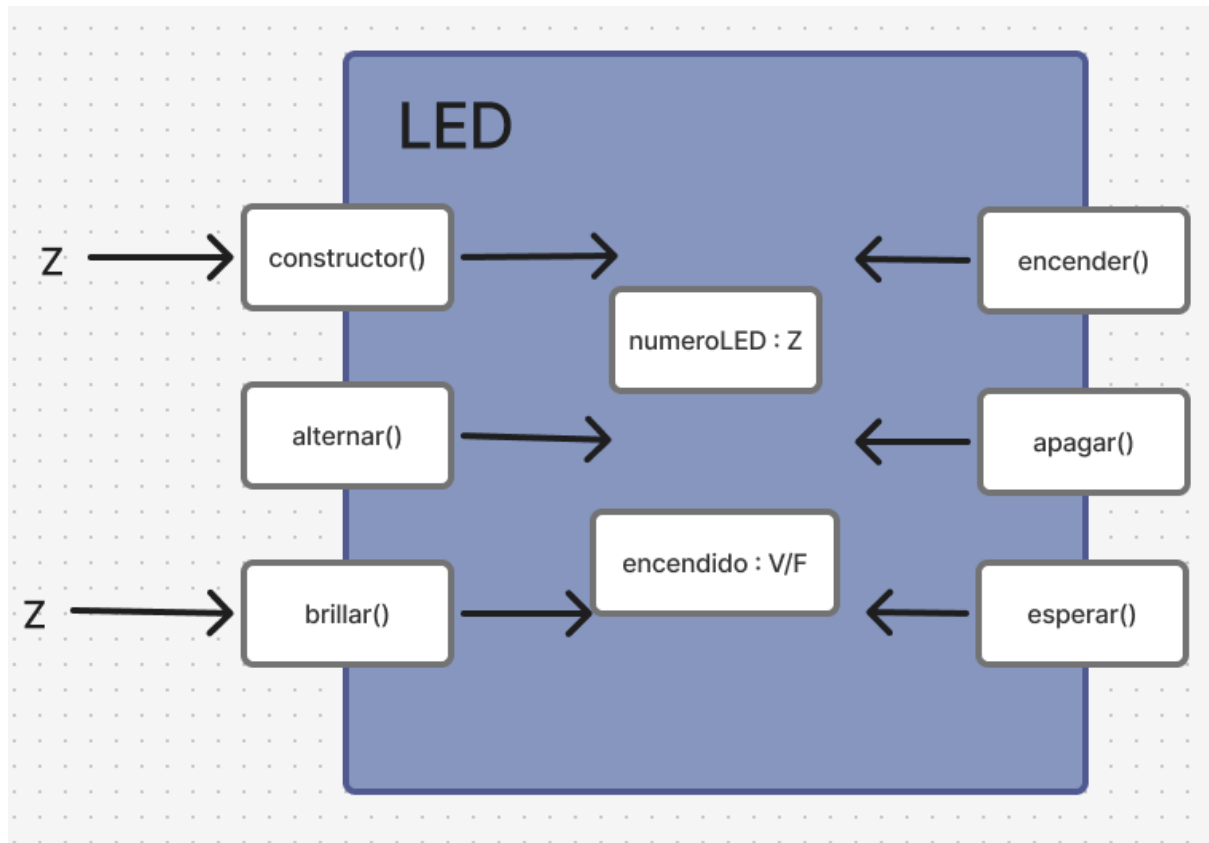
- **setup():** Inicializa el puerto serie, hardware, emisora BLE y medidor.
- **loop():**
 - Realiza una secuencia de parpadeo LED.

- Mide CO2 y temperatura usando Medidor.
- Publica ambas mediciones como iBeacon usando Publicador.
- Envía un paquete iBeacon con carga libre de prueba.
- Espera y detiene el anuncio BLE.

Relaciones:

Utiliza instancias globales de LED, PuertoSerie, Publicador y Medidor.

2.1.2. LED.h



Propósito:

Controla un LED conectado a un pin digital.

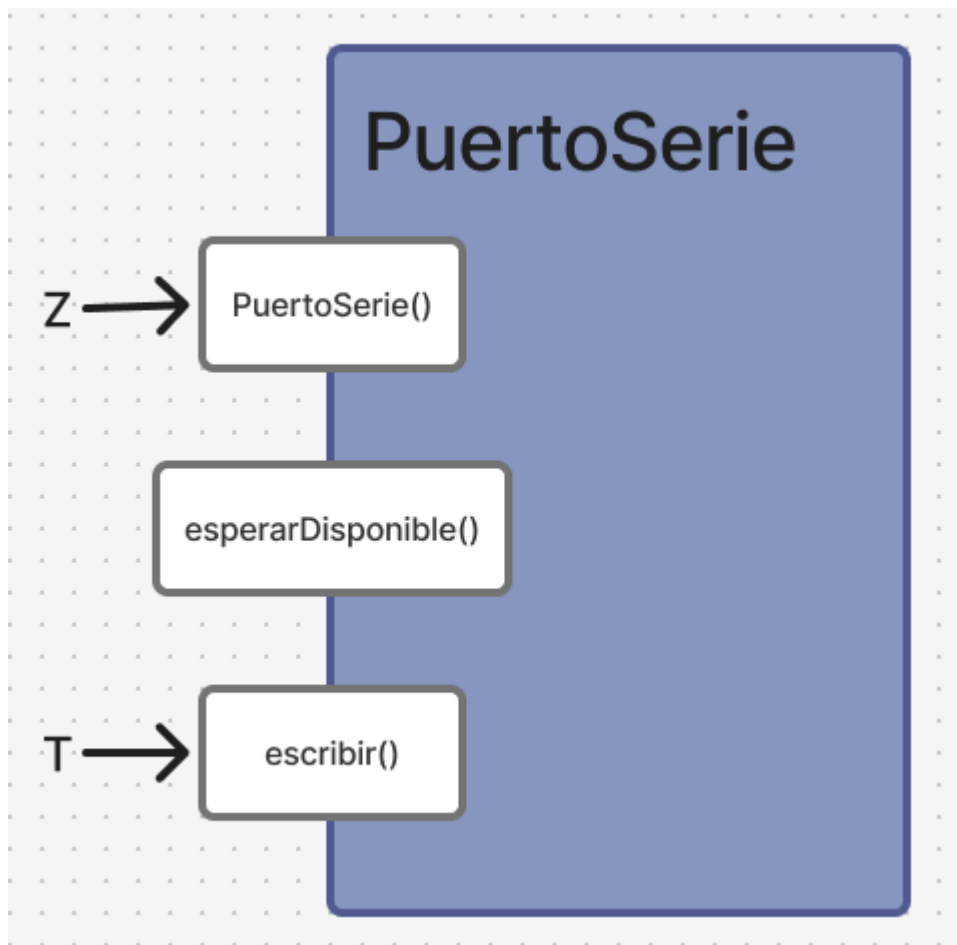
Funciones principales:

- LED(int numero): Constructor, inicializa el pin.
- encender(), apagar(), alternar(), brillar(long tiempo): Controlan el estado del LED.
- esperar(long tiempo): Función global para pausar la ejecución.

Relaciones:

Usado por el archivo principal para indicar actividad visual.

2.1.3. PuertoSerie.h



Propósito:

Abstracción para la comunicación serie.

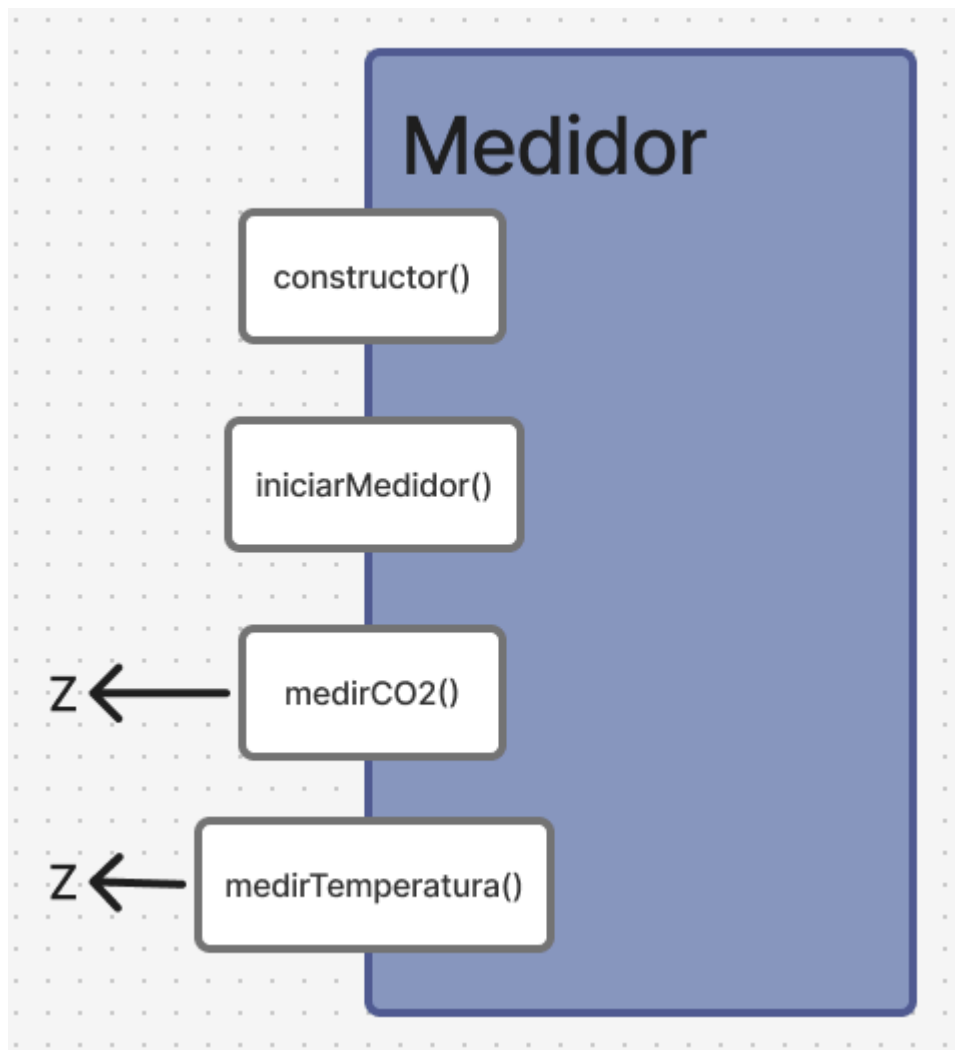
Funciones principales:

- `PuertoSerie(long baudios)`: Inicializa la comunicación serie.
- `esperarDisponible()`: Espera a que el puerto serie esté listo.
- `escribir(T mensaje)`: Imprime mensajes por el puerto serie.

Relaciones:

Utilizado para depuración y mensajes de estado en todo el sistema.

2.1.4. Medidor.h



Propósito:

Simula la medición de variables ambientales (CO2 y temperatura).

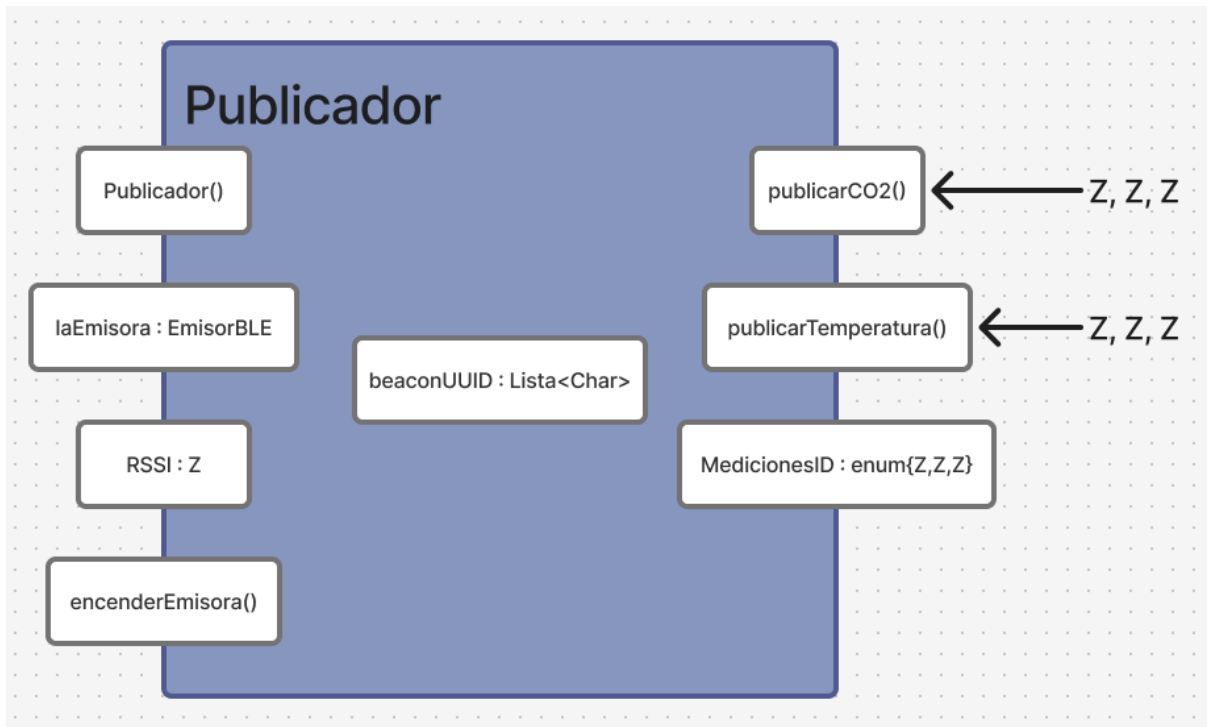
Funciones principales:

- `iniciarMedidor()`: Inicialización (vacía).
- `medirCO2()`: Devuelve un valor fijo de CO2.
- `medirTemperatura()`: Devuelve un valor fijo de temperatura.

Relaciones:

Llamado desde el archivo principal para obtener valores a publicar.

2.1.5. Publicador.h



Propósito:

Gestiona la publicación de mediciones como paquetes iBeacon vía BLE.

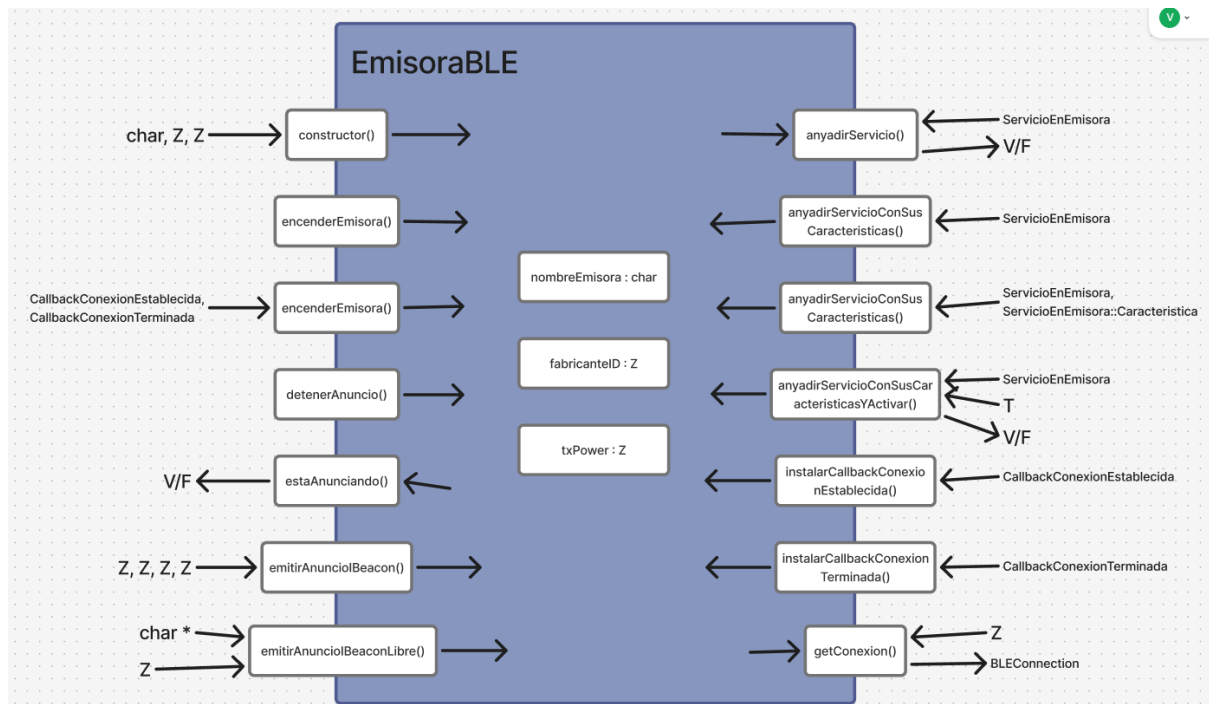
Funciones principales:

- **encenderEmisora()**: Inicializa la emisora BLE.
- **publicarCO2()**, **publicarTemperatura()**: Publican los valores recibidos como iBeacon usando EmisorBLE.

Relaciones:

Contiene una instancia de EmisorBLE. Usado por el archivo principal para enviar datos.

2.1.6. EmisoraBLE.h



Propósito:

Abstracción de la emisora Bluetooth LE, permite emitir anuncios iBeacon y gestionar servicios/características BLE.

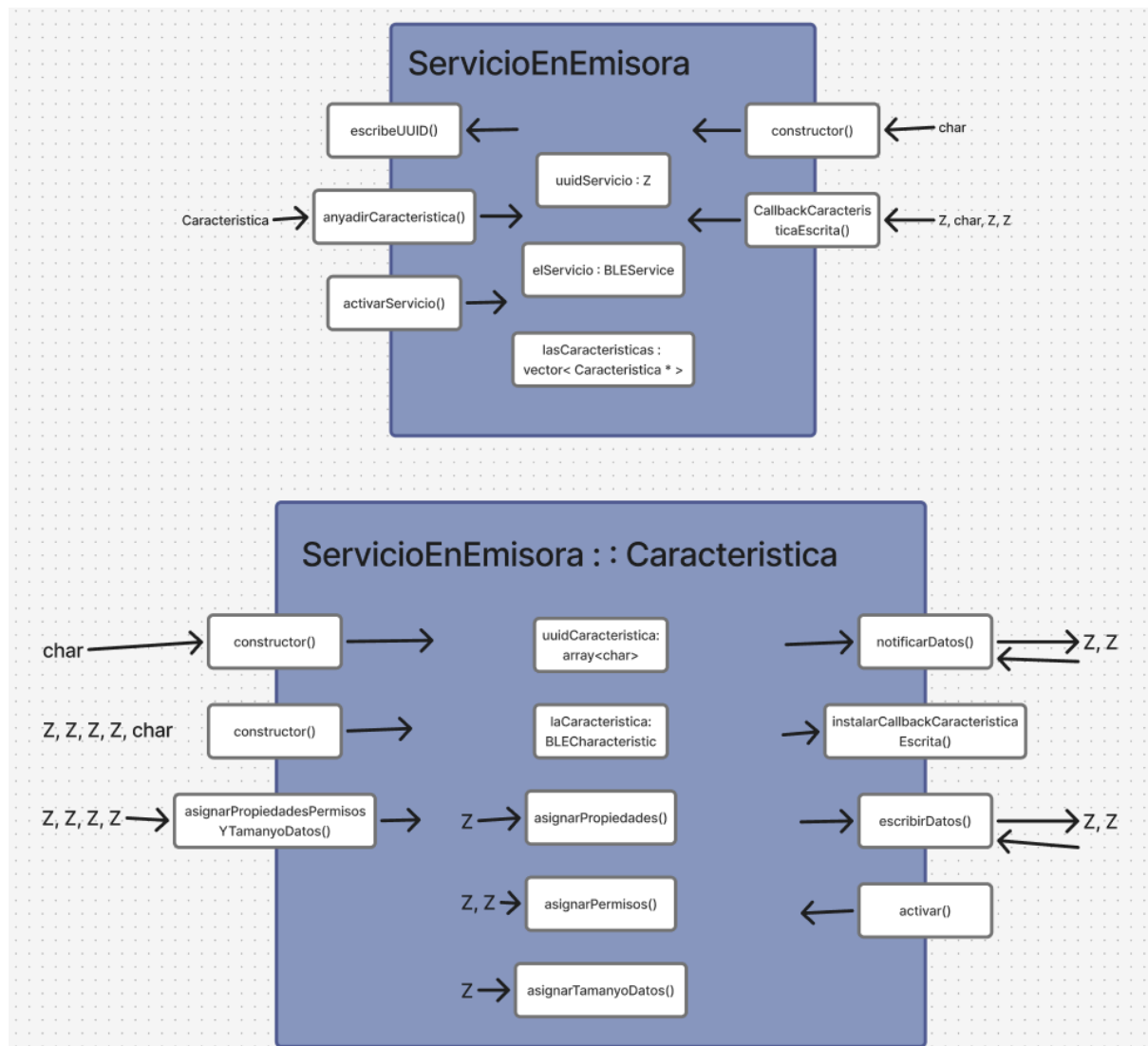
Funciones principales:

- `encenderEmisora()`, `detenerAnuncio()`, `estaAnunciando()`: Controlan el estado de la emisora.
- `emitirAnuncioBeacon()`: Emite un paquete iBeacon estándar.
- `emitirAnuncioBeaconLibre()`: Emite un paquete iBeacon con carga arbitraria.
- Métodos para añadir servicios/características BLE y callbacks de conexión.

Relaciones:

Utiliza `ServicioEnEmisora` para la gestión avanzada de servicios/características BLE. Usado por Publicador.

2.1.7. ServicioEnEmisora.h



Propósito:

Abstracción para definir servicios y características BLE personalizados.

Funciones principales:

- ServicioEnEmisora: Define un servicio BLE con UUID.
- Caracteristica: Clase anidada para definir características BLE.
- Métodos para añadir características, activar servicios y gestionar callbacks.

Relaciones:

Utilizado por EmisoraBLE para la gestión avanzada de servicios BLE.

2.1.8 Resumen de relaciones

- HolaMundoIBeacon.ino es el punto de entrada y orquestador.
- Utiliza LED para feedback visual y PuertoSerie para depuración.
- Obtiene mediciones de Medidor.

- Publica los datos usando Publicador, que a su vez usa EmisoraBLE.
- EmisoraBLE puede gestionar servicios/características BLE personalizados mediante ServicioEnEmisora.

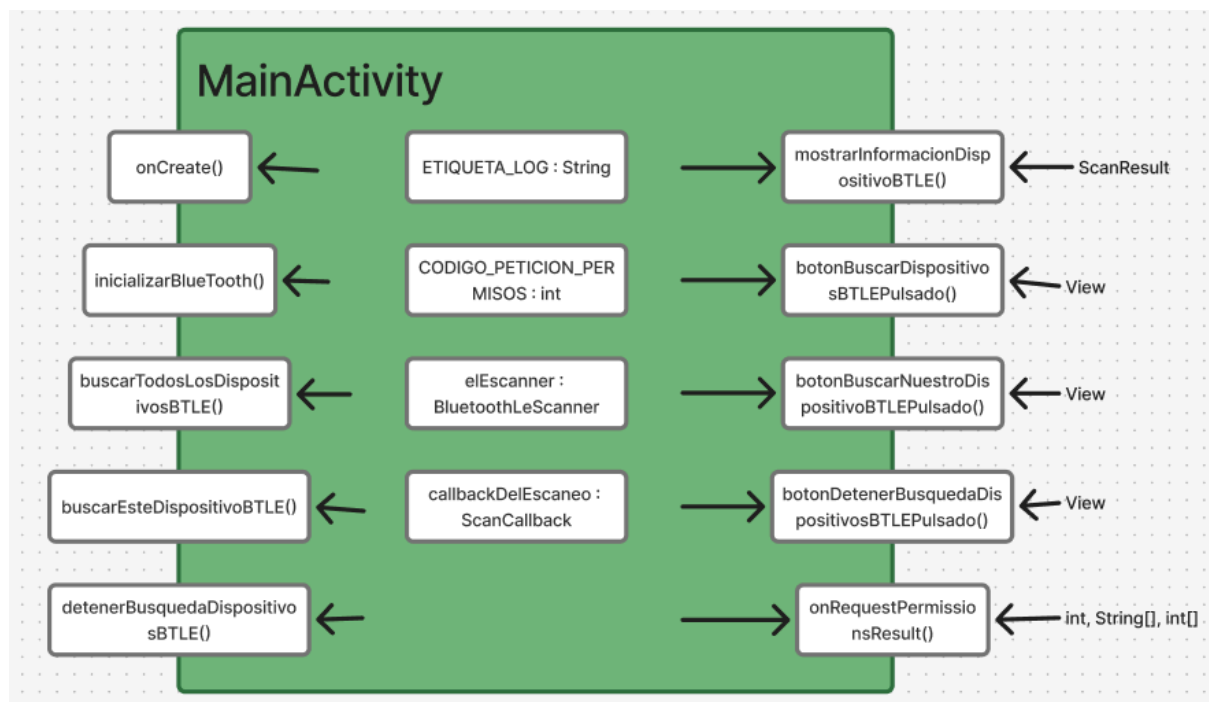
Flujo principal:

Medición → Publicador → EmisoraBLE → Bluetooth LE (iBeacon)

Soporte: LED y PuertoSerie para feedback y depuración.

2.2 Android

2.2.1 MainActivity.java



Propósito:

Actividad principal de la app. Gestiona la inicialización de Bluetooth, la petición de permisos y el escaneo de dispositivos Bluetooth Low Energy (BLE), mostrando información detallada de los paquetes iBeacon detectados.

Funciones principales:

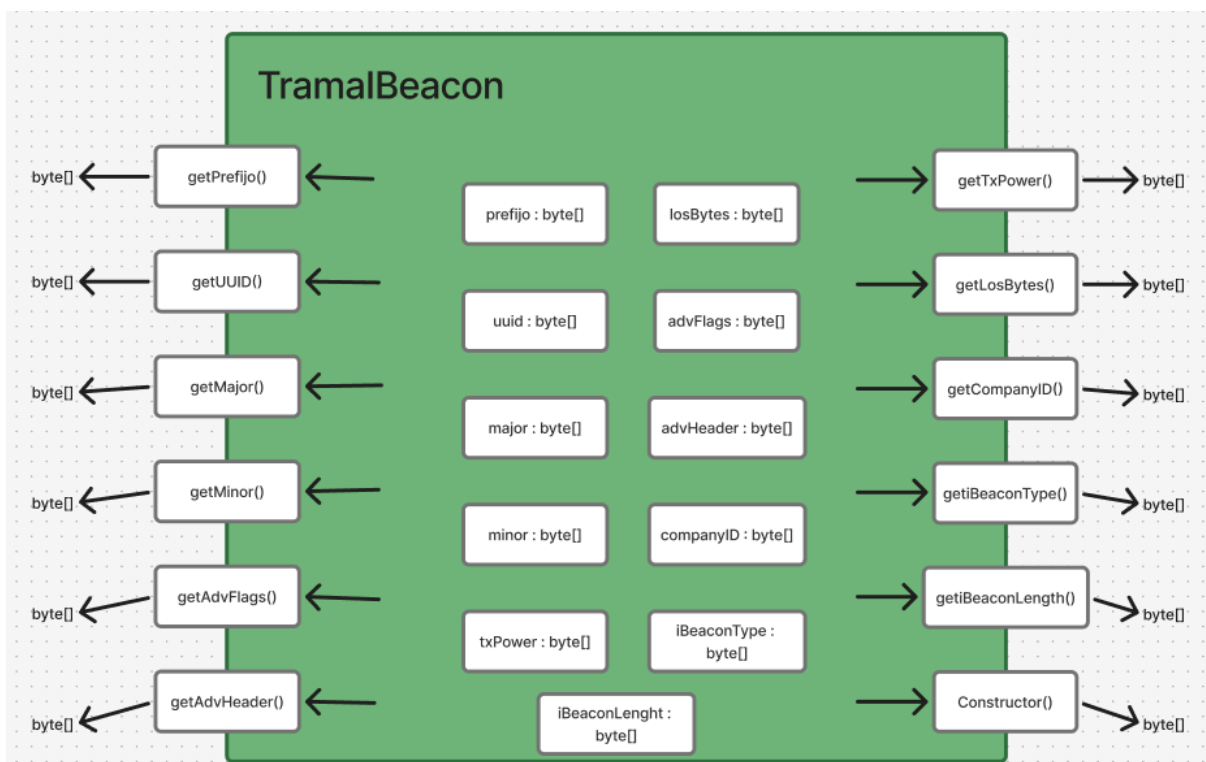
- **onCreate():** Inicializa la interfaz y Bluetooth, solicita permisos si es necesario.
- **inicializarBluetooth():** Obtiene el adaptador Bluetooth, lo habilita y prepara el escáner BLE.
- **buscarTodosLosDispositivosBTLE():** Inicia el escaneo BLE para todos los dispositivos, instalando un callback para procesar los resultados.

- **buscarEsteDispositivoBTLE(String nombre):** Inicia el escaneo BLE filtrando por nombre de dispositivo.
- **detenerBusquedaDispositivosBTLE():** Detiene el escaneo BLE.
- **mostrarInformacionDispositivoBTLE(ScanResult):** Extrae y muestra información relevante del dispositivo BLE detectado, incluyendo el análisis de la trama iBeacon usando TramaIBeacon.
- **Botones UI:** Métodos públicos para asociar a botones de la interfaz y lanzar las búsquedas o detenerlas.
- **onRequestPermissionsResult():** Gestiona la respuesta a la petición de permisos.

Relaciones:

- Usa TramaIBeacon para analizar los bytes de la trama iBeacon.
- Usa Utilidades para conversión de datos (bytes, hex, UUID, etc.).

2.2.2 TramaIBeacon.java



Propósito:

Clase de análisis de tramas iBeacon. Extrae los campos estándar de un paquete iBeacon recibido como array de bytes.

Campos principales:

- `prefijo`: 9 bytes iniciales del paquete iBeacon.
- `uuid`: 16 bytes de UUID.
- `major`, `minor`: 2 bytes cada uno, identificadores adicionales.
- `txPower`: 1 byte, potencia de transmisión.
- `advFlags`, `advHeader`, `companyID`, `iBeaconType`, `iBeaconLength`: Subcampos del prefijo.

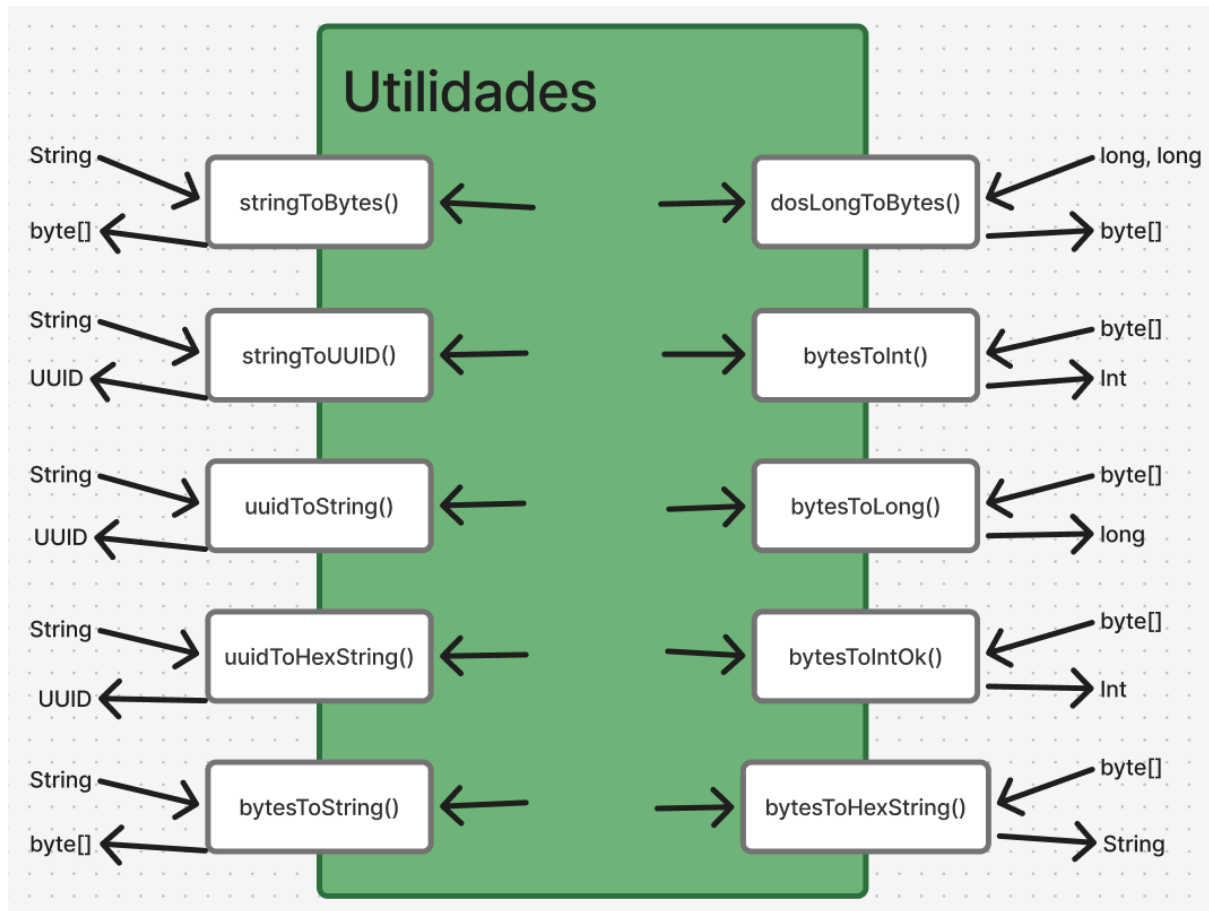
Métodos principales:

- Constructor: Recibe el array de bytes y extrae los campos según las posiciones estándar de iBeacon.
- Getters para cada campo.

Relaciones:

- Usado por `MainActivity` para descomponer y mostrar la información de cada paquete iBeacon detectado.

2.2.3 Utilidades.java



Propósito:

Clase de utilidades para conversión y manipulación de datos binarios y UUIDs.

Funciones principales:

- `stringToBytes(String)`: Convierte un string a array de bytes.
- `stringToUUID(String)`: Convierte un string de 16 caracteres a un objeto UUID.
- `uuidToString(UUID)`, `uuidToHexString(UUID)`: Convierte UUID a string o a hexadecimal.
- `bytesToString(byte[])`, `bytesToHexString(byte[])`: Convierte arrays de bytes a string o hexadecimal.
- `dosLongToBytes(long, long)`: Junta dos long en un array de bytes.
- `bytesToInt(byte[])`, `bytesToLong(byte[])`: Convierte arrays de bytes a int o long.

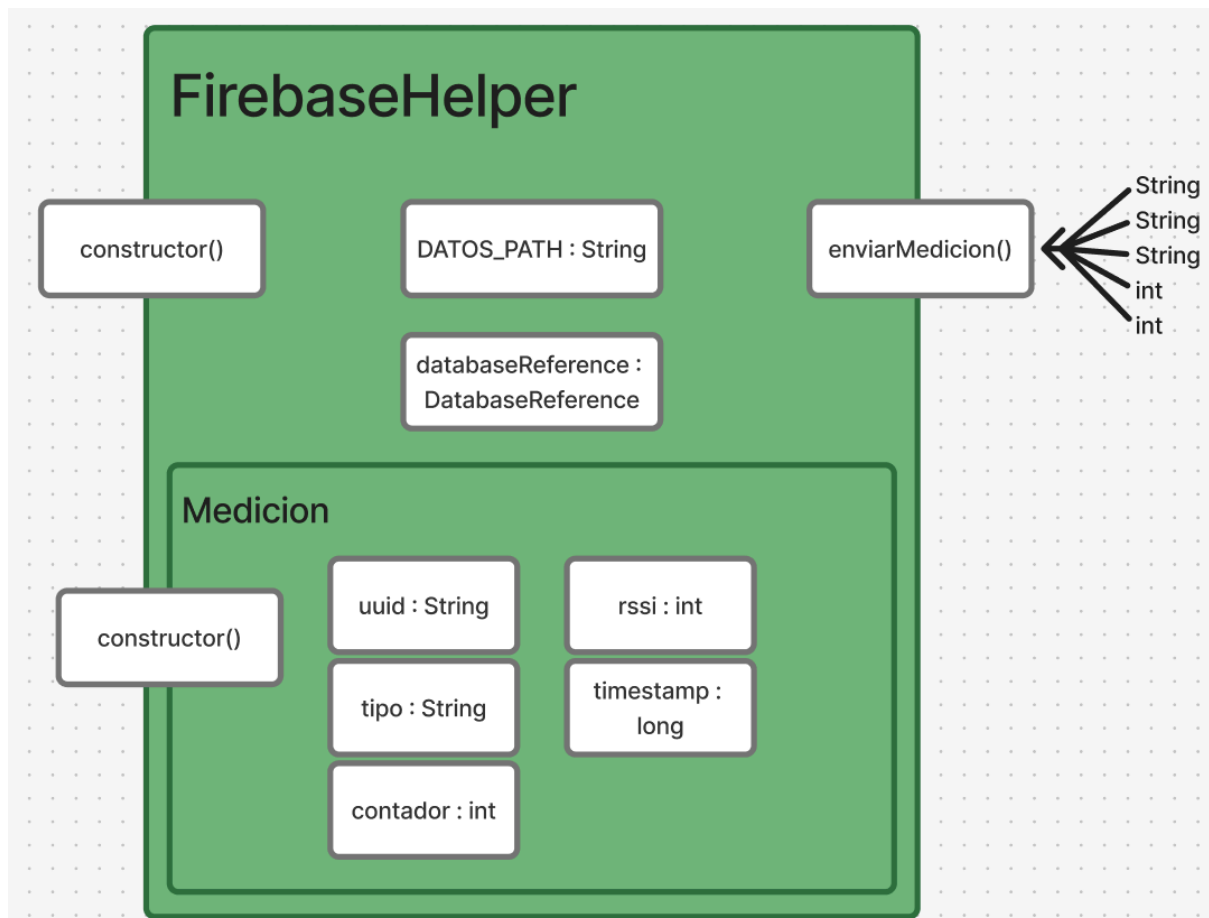
- `bytesToInt0K(byte[])`: Conversión de bytes a int considerando el signo.

Relaciones:

- Usado por `MainActivity` y `TramaIBeacon` para mostrar y manipular datos recibidos de los dispositivos BLE.

2.2.4 Lógica Fake (FirebaseHelper.java)

El archivo `FirebaseHelper.java` corresponde a la lógica fake de la aplicación. Se encarga de almacenar y subir el dato de la medición al backend, desarrollado con Firebase.



Propósito:

Encapsula la lógica de comunicación entre la app Android y Firebase Realtime Database, permitiendo registrar mediciones ambientales en la base de datos de forma estructurada y segura.

Estructura principal:

1. Atributos

- **DATOS_PATH:**
Cadena constante "mediciones". Define la ruta en la base de datos donde se almacenan los datos.

- **databaseReference:**
Referencia a la rama "mediciones" en Firebase Realtime Database.
2. Constructor
 - Inicializa databaseReference usando la instancia global de Firebase y la ruta definida.
 3. Método público
 - **enviarMedicion(String uuid, String tipo, int contador, int valor, int rssi)**
 - Crea un objeto Medicion con los parámetros recibidos y el timestamp actual.
 - Envía el objeto a Firebase usando `push().setValue(medicion)`, lo que genera una clave única para cada medición.
 - Añade listeners para loguear el éxito o el error del envío.
 4. Clase interna estática: Medicion
 - Representa la estructura de cada medición enviada.
 - Campos:
 - **uuid:** Identificador del dispositivo.
 - **tipo:** Tipo de medición (ej. "CO2", "Temperatura").
 - **contador:** Número secuencial de la medición.
 - **valor:** Valor medido.
 - **rssi:** Intensidad de la señal.
 - **timestamp:** Fecha/hora en milisegundos.
 - Incluye constructor vacío (requerido por Firebase) y constructor completo.

Lógica de funcionamiento:

1. Inicialización:
Al crear un objeto **FirebaseHelper**, se prepara la referencia a la rama "mediciones" de la base de datos.
2. Envío de datos:
Cuando se llama a **enviarMedicion**, se crea un objeto **Medicion** con los datos y se envía a Firebase.
El método es asíncrono y registra en el log si el envío fue exitoso o si hubo error.
3. Estructura en Firebase:
Cada medición se almacena como un hijo bajo "mediciones", con una clave única generada por `push()`.

Relación con el resto de la app:

- Este helper es invocado desde la lógica principal (por ejemplo, desde **MainActivity**) cada vez que se detecta y filtra una nueva medición válida.
- Permite desacoplar la lógica de negocio de la lógica de persistencia.

Resumen de relaciones y flujo

- **MainActivity** es el punto de entrada y orquestador de la app.
- Utiliza **Utilidades** para convertir y mostrar datos binarios y UUIDs.

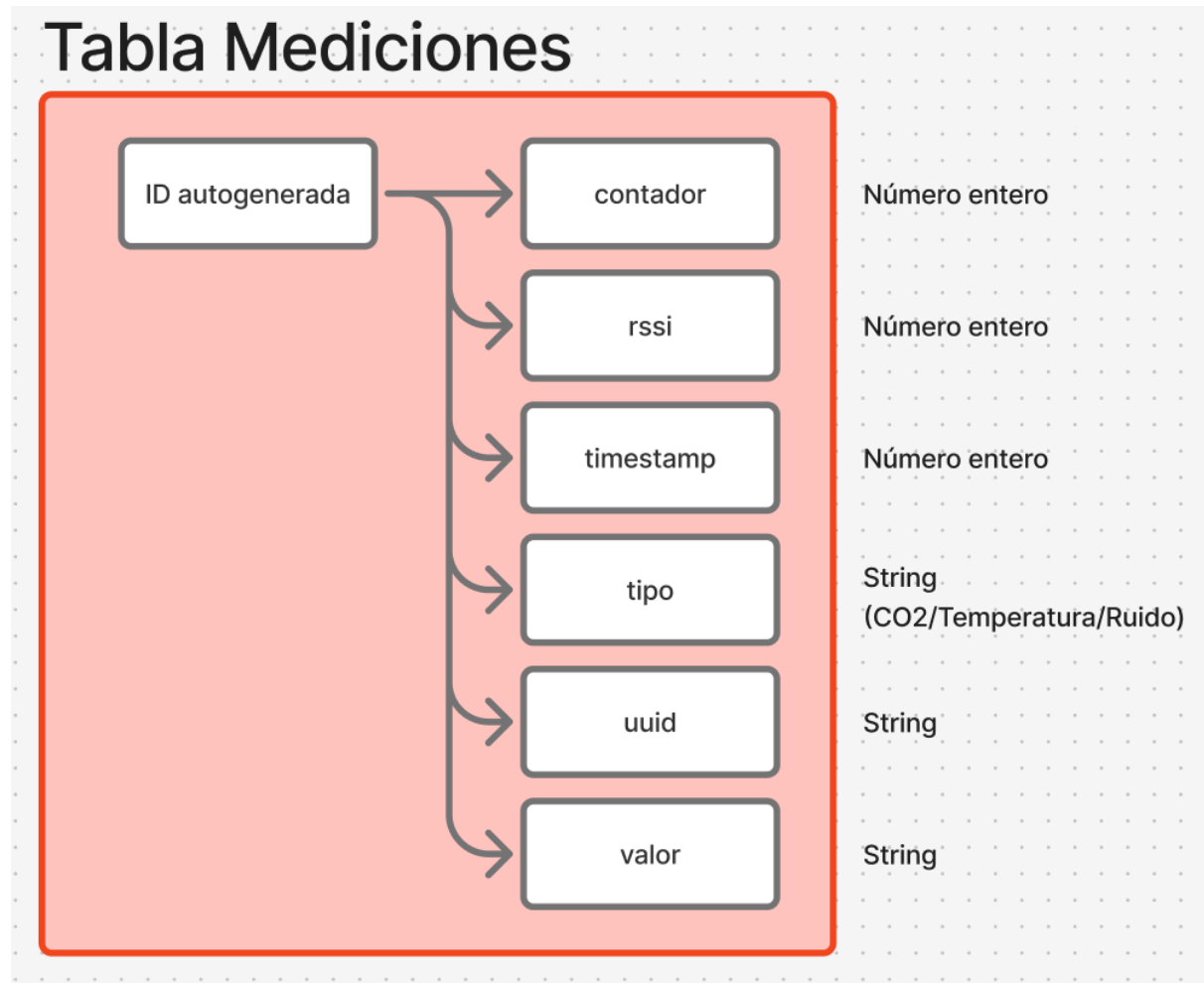
- Utiliza **TramalBeacon** para analizar los paquetes iBeacon detectados y extraer sus campos.
- **FirebaseHelper** se encarga de subir los datos de la medición a Firebase.
- El usuario puede iniciar el escaneo BLE (todos los dispositivos o filtrando por nombre) y detenerlo desde la interfaz.
- Cada vez que se detecta un dispositivo, se muestra información detallada de la trama iBeacon recibida.

Flujo principal:

Usuario pulsa botón → MainActivity inicia escaneo BLE → Recibe ScanResult → Analiza bytes con TramalBeacon → Muestra información usando Utilidades → FirebaseHelper lo envía al backend.

3. Diseño BBDD

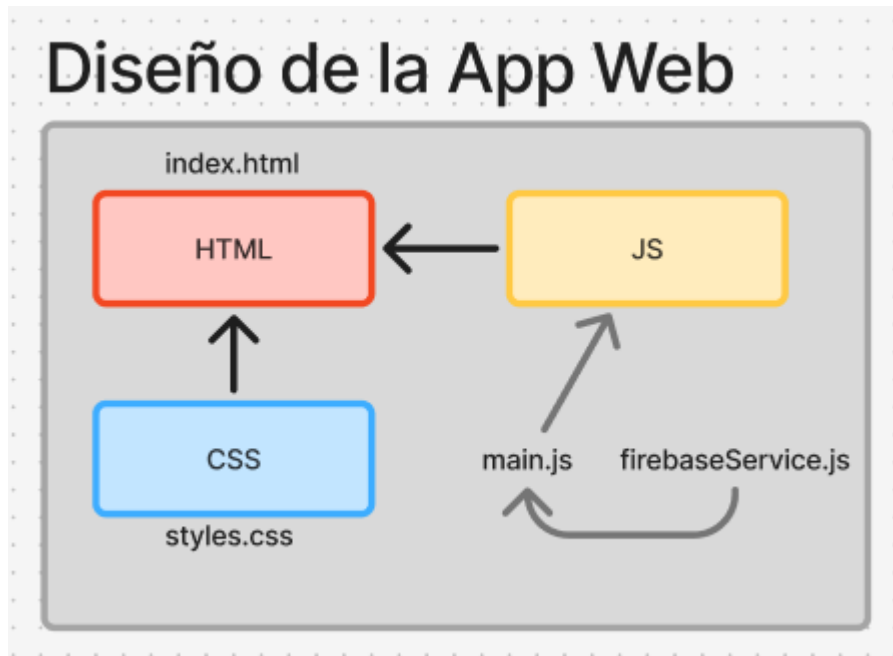
Se ha hecho uso de Firebase Realtime Database, una base de datos no SQL. Se dispone de una tabla Mediciones con valores compuestos por una ID autogenerada y un objeto con los siguientes componentes:



- **contador:** número entero
Corresponde al número de la medición tomada en ejecución.
- **rssi:** número entero
- **timestamp:** número
Fecha y hora en la que se ha enviado la medición.
- **tipo:** string
Tipo de medición a la que corresponde (CO2 / Temperatura / Ruido).
- **uuid:** string
- **valor:** string
El valor que ha dado la medición.

4. Web

4.1 Diseño de la App Web



La estructura de la Aplicación Web está compuesta por:

- I. **HTML**: Esqueleto de la aplicación que hace uso del resto de archivos como recursos.
- II. **CSS**: Hoja de estilos que da un aspecto visual a la página.
- III. **JS**: Es la lógica de la aplicación y está compuesto por dos archivos:
 - A. **firebaseService.js**: Se encarga de hacer la petición de los datos a Firebase. Es la [lógica de negocio](#).
 - B. **main.js**: Llama a firebaseService y muestra los datos recibidos.

4.2 Visual de la App Web

Proyecto Aplicaciones de Biometría y Medio Ambiente

Cliente Web

Ultima medición:

Contador	1
Tipo	CO2
Valor	235
RSSI	-45 dBm
Fecha	12/10/2025, 20:05:52

Historial de mediciones:

Contador	Tipo	Valor	RSSI	Fecha
1	CO2	235	-45 dBm	12/10/2025, 20:05:52
1	CO2	235	-45 dBm	12/10/2025, 20:00:31
1	CO2	235	-45 dBm	12/10/2025, 19:08:45
1	Temperatura	20	-65 dBm	8/10/2025, 17:39:42
1	Temperatura	20	-65 dBm	8/10/2025, 17:39:40
1	Temperatura	20	-65 dBm	8/10/2025, 17:39:39
1	Temperatura	20	-65 dBm	8/10/2025, 17:39:39
1	Temperatura	20	-65 dBm	8/10/2025, 17:39:38

Víctor Morant Faus - 2025

Se han desarrollado dos secciones que permiten ver la última medición y un historial más detallado de las últimas mediciones.

5. GitHub

Se ha creado un repositorio público con enlace <https://github.com/victormnt37/Proyecto-Biometria> .

La estructura de carpetas es la siguiente:

```
doc/
  BiometriaSprint0.pdf
src/
  Arduino/
  cliente/
  telefono/
  servidor/
README.md
```

6. Trello

Este es el trello en el que se han establecido las tareas. <https://trello.com/b/IKULtAKQ/biometria-sprint-0>

7. Pruebas del funcionamiento

7.1 Tests automáticos

En la carpeta test/ se encuentra el archivo `firebaseService.test.js`. Este archivo contiene pruebas unitarias para el módulo de servicios de consultas a Firebase Realtime Database, asegurando el correcto funcionamiento de las funciones de obtención de mediciones en la aplicación web.

Objetivo de los tests

- Verificar que la función `getUltimaMedicion` obtiene correctamente la última medición registrada en la base de datos, según el campo `timestamp`.
- Comprobar que la función `getHistorialMediciones` recupera un número determinado de mediciones, ordenadas de forma descendente por fecha.

Estructura de las pruebas

1. Preparación del entorno de pruebas
 - Se utiliza la librería `firebase-mock` para simular una instancia de Firebase Realtime Database, evitando dependencias externas y permitiendo pruebas rápidas y controladas.
 - Antes de cada test, se cargan tres mediciones de ejemplo en la base de datos mock, cada una con valores distintos de contador, rssi, timestamp y valor.
2. Test 1: Última medición
 - Se llama a `getUltimaMedicion` y se comprueba que el valor y el timestamp corresponden a la medición más reciente (la de mayor timestamp).
3. Test 2: Historial de mediciones
 - Se llama a `getHistorialMediciones` solicitando las dos últimas mediciones.
 - Se verifica que se obtienen dos elementos y que están ordenados de forma descendente por el campo `timestamp`.

7.2 Flujo de funcionamiento



1. Ejecución **Arduino**
 - a. Se compila el código en la **Placa**.
 - b. Se envía el dato vía bluetooth.
2. Ejecución **Android Studio**
 - a. Desde la aplicación móvil **Android** se lee el dato recibido a través del IBeacon.
 - b. Se traducen el mayor y menor y se establece la fecha y hora de recepción.
 - c. Se envía a través de Firebase.
3. Ejecución **VS Code Live Server**
 - a. Desde un Navegador **Web** se ejecuta la aplicación que solicita los datos a Firebase.
 - b. El dato se actualiza al instante ante cualquier cambio gracias al servicio Realtime Database de Firebase.

En la imagen se puede ver el proyecto en su totalidad en ejecución.